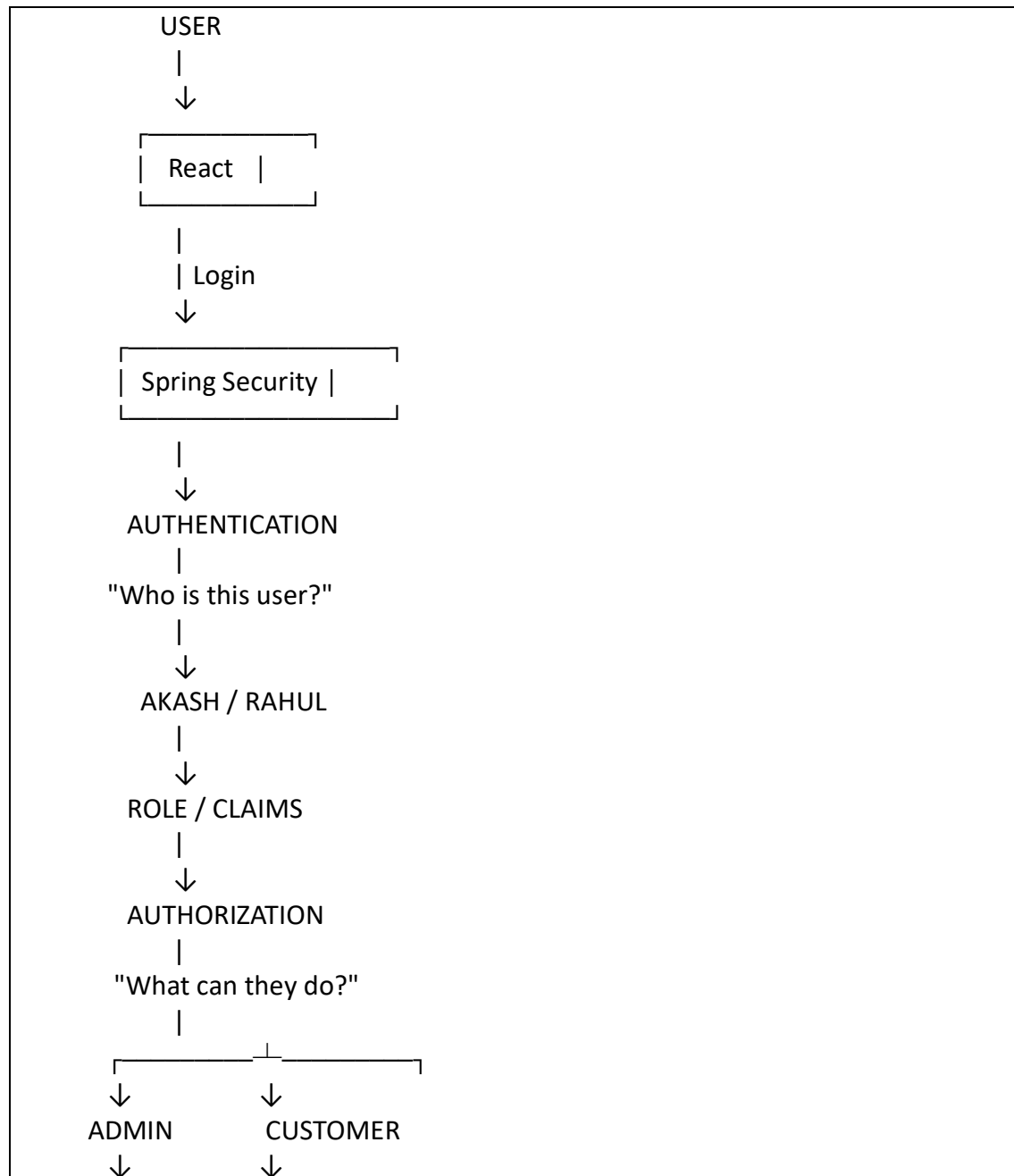


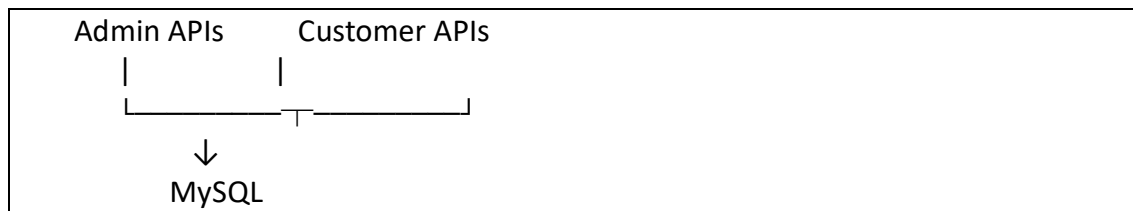
NAME : AKASH SAMPATH

Module : Research Questions_P3

Authentication : Authorization : [No authentication : Basic Authentication : Social /Open Authentication]

Architecture :

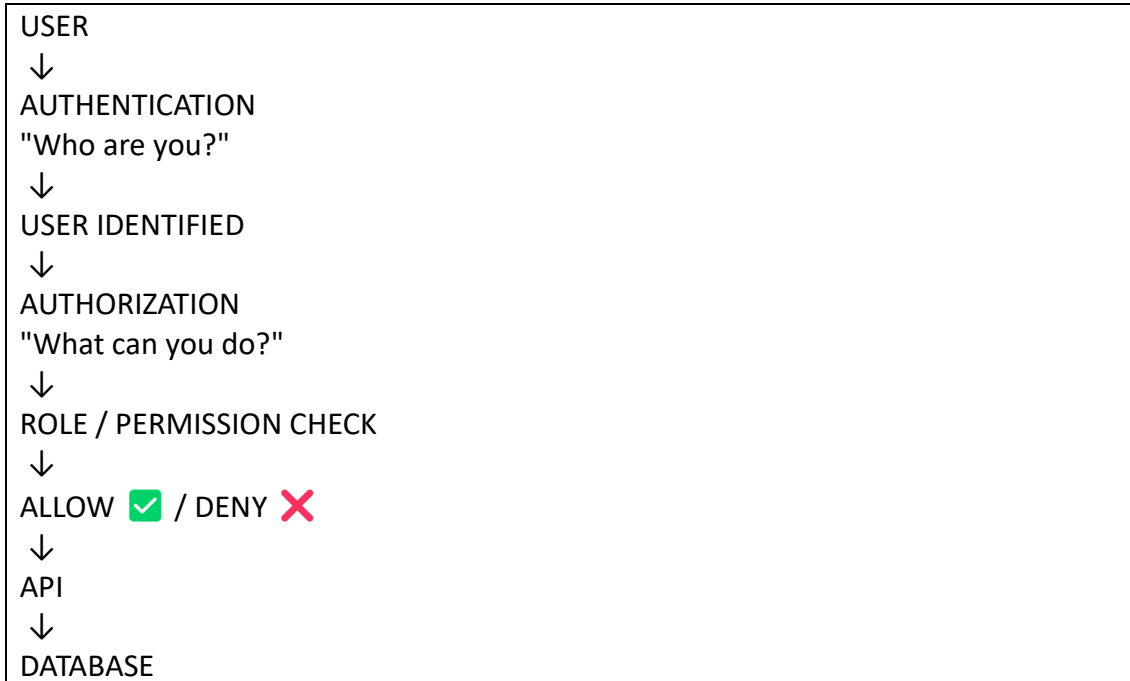




	Topic	Very Simple Meaning	Example
1	Authentication	Who are you?	Login with username/password
2	Authorization	What can you do?	Admin can delete bills
3	No Authentication	No login required	Anyone can call /api/customers
4	Basic Authentication	Login with username + password	akash + password
5	Social/Open Authentication	Login using another provider	Login with Google
6	OAuth 2.0	Framework used for delegated access	Google OAuth
7	OpenID Connect (OIDC)	Identity/authentication layer on OAuth 2.0	Google identifies the user
8	Spring Security	Security framework for Spring Boot	Protect APIs
9	User	Person using your application	Akash
10	Role	Defines what a user can do	ADMIN, EMPLOYEE, CUSTOMER
11	RBAC	Authorization based on roles	ADMIN → manage users
12	ADMIN	High-level access	Create/delete/manage
13	EMPLOYEE	Limited business access	Create/update bills
14	CUSTOMER	Normal user access	View own bills
15	Protected API	API requiring login	GET /api/profile
16	Role-Protected API	API requiring a specific role	/api/admin/**

17	JWT	Token used to represent authenticated identity/claims	Authorization: Bearer <token>
18	401	Not authenticated	User hasn't provided valid authentication
19	403	Authenticated but not allowed	CUSTOMER accessing ADMIN API
20	Frontend Security	Hide/show UI features	Hide Delete button
21	Backend Security	Actually enforce security	Spring Security rejects unauthorized API requests
22	Resource Authorization	Check access to a specific record	Customer can access only their own bill
23	Password Hashing	Safely store passwords	BCrypt / Argon2
24	Your Project	Electricity Bill Management System	React + Spring Boot + MySQL
25	Final Security Flow	Identify → Check permission → Allow/Deny	Login → Role → API → Database

Work Flow :



Remember :

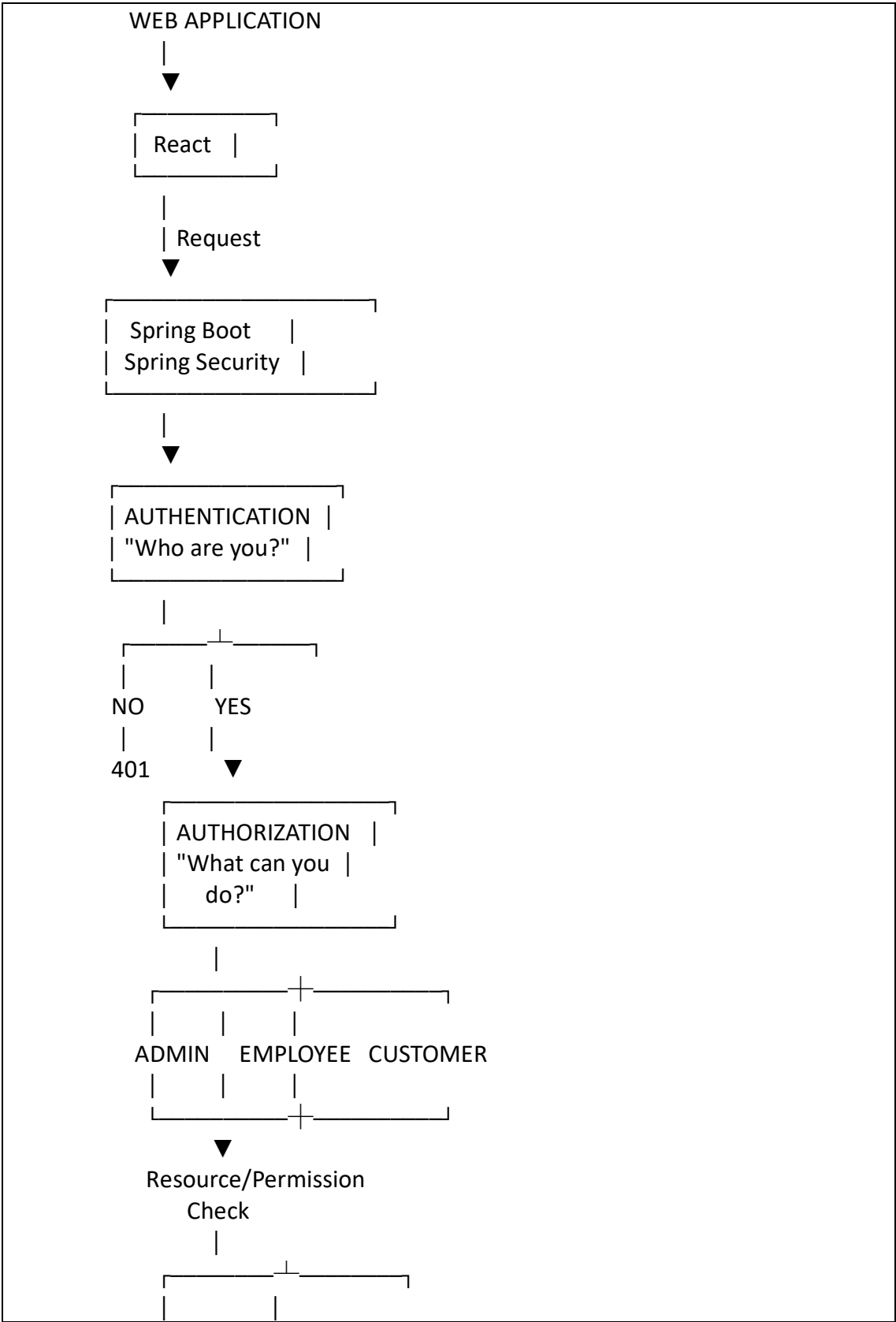
Question	Security Concept	Example
Who are you?	Authentication	Login with password / Google
Are you logged in?	Authentication	Session/JWT/OIDC
What role do you have?	Identity/Role	ADMIN
What can your role do?	Authorization	ADMIN can delete bills
Can you access this particular record?	Resource authorization	Customer can see only their own bill
Not logged in?	401	Authentication required
Logged in but not allowed?	403	CUSTOMER trying admin operation
Can React hide a button?	UI control	Hide Delete button
Is hiding the button security?	No	Backend must enforce it

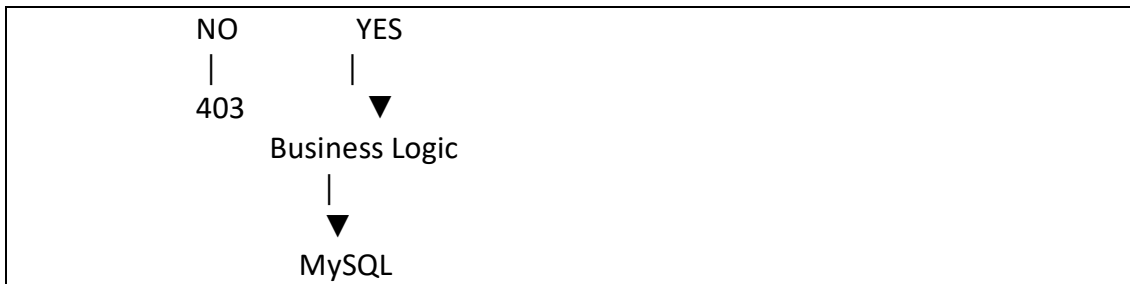
Where must authorization ultimately be enforced?	Backend	Spring Security
What should you learn first?	No Auth → Basic → Social/OIDC → Authorization	Progressive learning

Authentication vs Authorization :

	Authentication	Authorization
Main question	Who are you?	What can you do?
Happens	First	After authentication
Example	Login	Admin access
Technologies	Password, Google, OIDC	Roles, permissions, policies
Example result	Akash	ADMIN
Failure	401	403
Protects	Identity	Resources/actions

Whole concept :





What is Pydantic settings and when to use? What is the equivalent o Swagger in Python/FastAPI world?

FastAPI	Build API
Pydantic	Validate data
Pydantic Settings	Manage configuration
.env	Store configuration
OpenAPI	API blueprint
Swagger UI	See + test API
ReDoc	Alternative API documentation

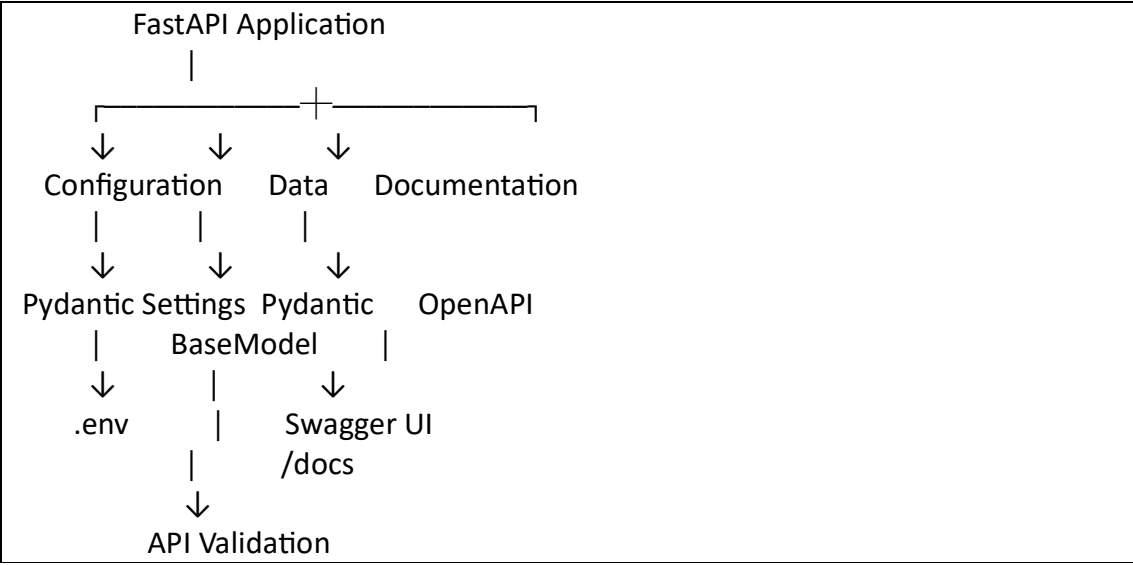
Spring boot vs Fast API :

Spring boot

FastAPI/Python

Spring Boot	FastAPI
application.properties	.env
@ConfigurationProperties	Pydantic Settings
@GetMapping	@app.get()
@PostMapping	@app.post()
@PutMapping	@app.put()
@DeleteMapping	@app.delete()
Spring validation	Pydantic validation

Architecture :



How Connects :

Step	Technology	Job
1	.env	Store configuration
2	Pydantic Settings	Read + validate configuration
3	FastAPI	Build API endpoints
4	Pydantic	Validate API request/response data
5	OpenAPI	Create API specification automatically
6	Swagger UI	Display and test APIs
7	ReDoc	Display API documentation another way

Topic	What it means	Simple example	Easy way to remember
FastAPI	Python framework for building APIs	<code>app = FastAPI()</code>	Build APIs
API	Allows frontend and backend to communicate	<code>GET /customers</code>	Communication
Pydantic	Validates and structures data	<code>class User(BaseModel)</code>	Validate data
Pydantic BaseModel	Defines request/response data format	<code>name: str, age: int</code>	Data model / DTO
Pydantic Settings	Manages application configuration	<code>DATABASE_URL, JWT_SECRET</code>	Manage configuration
<code>.env</code>	Stores configuration/environment values	<code>DATABASE_URL=...</code>	Configuration storage
Environment variables	Values provided to the application from the environment	<code>DATABASE_URL</code>	External configuration
OpenAPI	Standard description/specification of your API	<code>/openapi.json</code>	API blueprint
Swagger UI	Interactive website to view and test APIs	<code>/docs</code>	API testing screen
ReDoc	Another API documentation interface	<code>/redoc</code>	Alternative docs
FastAPI /docs	Automatically generated Swagger UI	<code>http://localhost:8000/docs</code>	Swagger
FastAPI /redoc	Automatically generated ReDoc	<code>http://localhost:8000/redoc</code>	ReDoc

FastAPI /openapi.json	Automatically generated OpenAPI specification	http://localhost:8000/openapi.json	API specification
GET	Retrieve data	@app.get("/customers")	Read
POST	Create/send data	@app.post("/customers")	Create
PUT	Update data	@app.put("/customers/{id}")	Update
DELETE	Delete data	@app.delete("/customers/{id}")	Delete
Request model	Defines what data the API accepts	Customer(BaseModel)	Input
Response model	Defines what API returns	response_model=Customer	Output
Validation	Checks whether received data is correct	age: int	Check data

Cheat Sheet :

Topic	Remember this
Pydantic	Validate data
Pydantic BaseModel	Define request/response data structure
Pydantic Settings	Manage application configuration
.env	Store environment-specific configuration
FastAPI	Build APIs using Python
OpenAPI	Standard API description/specification
Swagger UI	Interactive API documentation/testing UI
FastAPI /docs	Swagger UI
FastAPI /redoc	ReDoc
FastAPI /openapi.json	OpenAPI specification

Examples :

#	Topic	Real technical example	What happens	Easy memory
1	FastAPI	<code>app = FastAPI()</code>	Creates your Python web API application	Build API
2	API Endpoint	<code>@app.get("/customers")</code>	Creates an endpoint that clients can call	API URL
3	GET	<code>GET /customers</code>	Retrieves customers	Read
4	POST	<code>POST /customers</code>	Creates a customer	Create
5	PUT	<code>PUT /customers/1</code>	Updates customer 1	Update
6	DELETE	<code>DELETE /customers/1</code>	Deletes customer 1	Delete
7	Pydantic	<code>class Customer(BaseModel):</code>	Defines and validates API data	Validate data
8	Pydantic Model	<code>name: str, email: str, age: int</code>	Defines what data is expected	DTO / Data structure
9	Request Body	<code>{"name":"Akash","age":25}</code>	Client sends data to FastAPI	Input
10	Validation	<code>age: int</code> but user sends "hello"	FastAPI/Pydantic rejects invalid data	Check input
11	Response Model	<code>response_model=Customer</code>	Controls/validates returned data	Output structure
12	Pydantic Settings	<code>class Settings(BaseSettings):</code>	Reads and validates application configuration	Configuration manager
13	.env	<code>DATABASE_URL=...</code>	Stores environment-	Config storage

			specific configuration	
14	Environment Variable	DATABASE_URL	Application receives configuration externally	External config
15	Secret	JWT_SECRET=...	Keeps sensitive configuration outside source code	Don't hard-code secrets
16	OpenAPI	/openapi.json	FastAPI automatically generates API specification	API blueprint
17	Swagger UI	/docs	Interactive page for viewing/testing APIs	Test API visually
18	ReDoc	/redoc	Alternative API documentation page	Another API docs UI
19	Uvicorn	uvicorn main:app --reload	Runs your FastAPI application	Run server
20	Database	MySQL/PostgreSQL	Stores application data	Store data

Part	Example
Application	FastAPI
Database	MySQL
Configuration	.env
Configuration management	Pydantic Settings
Data validation	Pydantic
API documentation	Swagger UI
API specification	OpenAPI
API server	Uvicorn

Remember :

Remember	Meaning
FastAPI	Build the API
Pydantic	Validate the data
Pydantic Settings	Manage configuration
.env	Store configuration
OpenAPI	Describe the API
Swagger UI	View + test the API
ReDoc	Alternative API documentation
Uvicorn	Run the FastAPI server
MySQL	Store application data

Are you using Hibernate in your Springboot app?

1. if not, which ORM are you using?
2. where are you mentioning the ORM details in your springbot app

What is Swagger or Open API*?

1. what is this used for? Explain me this with real technical examples to learn and understand very very easily

So Basically , React calls Spring Boot APIs → Spring Boot uses Spring Data JPA → Hibernate converts Java objects/operations into SQL → MySQL stores the data. Meanwhile, OpenAPI describes those APIs and Swagger UI lets you see and test them.

ORM	Java objects ↔ database tables
JPA	ORM specification/rules
Hibernate	Implements ORM/JPA
Spring Data JPA	Makes JPA database work easier
MySQL	Stores the actual data
OpenAPI	Describes your REST APIs
Swagger UI	Lets you view and test those APIs
Postman	Tool for testing APIs
Swagger/OpenAPI	Mainly concerned with REST API documentation/testing

	Topic	Very simple meaning	Your Spring Boot project
1	Spring Boot	Framework used to build Java backend applications	Yes
2	REST API	Allows frontend and backend to communicate using HTTP	GET, POST, PUT, DELETE
3	Controller	Receives API requests	@RestController

4	Service	Contains business logic	CustomerService etc.
5	Repository	Talks to database through JPA	JpaRepository
6	Database	Stores application data	MySQL
7	ORM	Maps Java objects ↔ database tables	Yes
8	JPA	Standard/rules for ORM in Java	Yes
9	Hibernate	ORM implementation that follows JPA	Yes — you are using Hibernate
10	Spring Data JPA	Makes JPA/Hibernate database work much easier	Yes
11	@Entity	Tells JPA that a Java class represents database data	Customer etc.
12	@Id	Defines the primary key	id
13	JpaRepository	Provides ready-made database operations	findAll(), save(), delete()
14	findAll()	Gets records from database	Instead of manually writing SQL
15	save()	Inserts/updates a record	Hibernate generates SQL
16	JDBC Driver	Allows Java/Spring Boot to connect to MySQL	MySQL Connector/J

17	spring.datasource.url	Tells Spring Boot where the database is	Your MySQL/RDS URL
18	spring.datasource.username	Database username	Environment variable
19	spring.datasource.password	Database password	Environment variable
20	spring.jpa.hibernate.ddl-auto	Controls how Hibernate handles database tables/schema	You use update
21	OpenAPI	Standard format for describing REST APIs	Documents your APIs
22	Swagger	Collection of tools built around OpenAPI	Used with your APIs
23	Swagger UI	Web page showing your APIs	Lets you view/test APIs
24	GET	Retrieve data	GET /api/customers
25	POST	Create data	POST /api/customers
26	PUT	Update data	PUT /api/customers/{id}
27	DELETE	Delete data	DELETE /api/customers/{id}
28	Try it out	Swagger UI button for testing an API	Send API request directly
29	Postman	Tool for testing APIs	Alternative/complement to Swagger
30	Authentication	"Who are you?"	Login/JWT/OAuth2
31	Authorization	"What are you allowed to do?"	Roles/permissions
32	OpenAPI + Authentication	Can document how APIs require authentication	JWT/OAuth2 can be documented

Layer	Technology	Job
Frontend	React	User interface
↓	HTTP/REST	Communicates with backend
Backend	Spring Boot	Runs application logic
↓	Controller	Receives API request
↓	Service	Business logic
↓	Spring Data JPA	Database abstraction
↓	Hibernate	ORM — Java objects ↔ database
↓	JDBC/MySQL Driver	Database connection
Database	MySQL	Stores data

And alongside the backend:

Technology	Job
OpenAPI	Describes your REST APIs
Swagger UI	Displays and lets you test those APIs

Important :

If someone asks...	Answer
Which database?	MySQL
Which ORM?	Hibernate
Which ORM standard?	JPA
How do you conveniently use JPA?	Spring Data JPA
What maps Java objects to DB tables?	Hibernate/JPA
What describes your REST APIs?	OpenAPI
What UI can display/test those APIs?	Swagger UI
What is an alternative API testing tool?	Postman

Step	Technology / Code	What happens	Easy example
1	React	User enters customer details	Name = Akash, Email = akash@gmail.com
2	REST API	React sends request to backend	POST /api/customers
3	Controller	Receives the request	CustomerController
4	Service	Performs business logic	customerService.createCustomer()
5	Entity + JPA	Java class represents DB table	@Entity Customer
6	Spring Data JPA	Provides database methods	customerRepository.save(customer)
7	Hibernate	Converts Java object into SQL	INSERT INTO customers ...
8	JDBC Driver	Connects Spring Boot to MySQL	MySQL Connector/J
9	MySQL	Stores the customer	1, Akash, akash@gmail.com
10	Hibernate	Converts DB data back into Java objects	MySQL row → Customer object
11	OpenAPI	Describes the API	POST /api/customers + request/response
12	Swagger UI	Shows the API in a browser	Click Try it out → Execute
13	Postman	Another way to test the API	Send POST /api/customers
14	Authentication	Checks who the user is	Login

15	Authorization	Checks what the user can do	Admin can delete customers
----	----------------------	-----------------------------	----------------------------

Part	Example
Java Entity	@Entity class Customer
Database table	customers
Java field	private String name;
Database column	name
Repository	CustomerRepository extends JpaRepository<Customer, Long>
Create customer	customerRepository.save(customer)
Get customers	customerRepository.findAll()
API	GET /api/customers
Create API	POST /api/customers
ORM	Hibernate
ORM standard	JPA
Database	MySQL
API description	OpenAPI
API documentation/testing UI	Swagger UI
API testing alternative	Postman

	Remember
1	MySQL = stores data
2	ORM = Java objects ↔ database tables
3	JPA = ORM rules/specification
4	Hibernate = implements ORM/JPA

5	Spring Data JPA = makes database operations easy
6	OpenAPI = describes your APIs
7	Swagger UI = displays and lets you test those APIs

1. What are boiler plate codes in general?
2. how is this related to your frameworks and libraries that you use for creating webapplications?

Framework	Building structure
Library	Ready-made tool
Boilerplate	Repetitive work
Business logic	Your actual idea/rules

Layer	Technology	What it does ?
Frontend	React	Creates the user interface
Frontend HTTP	Axios	Calls backend APIs
Backend	Spring Boot	Creates and runs the backend
API	Spring MVC	Handles HTTP requests

ORM	Hibernate/JPA	Connects Java objects with MySQL data
JSON	Jackson	Converts Java objects to/from JSON
Database	MySQL	Stores customers, bills, etc.
Your code	Business Logic	Calculates bills and implements your application's rules

Step	What happens?	Example from Electricity Bill App	Who helps?
1	Create web application	Electricity Bill Management System	You
2	Need frontend	Login page, customer page, bill page	React
3	Need HTTP communication	React asks backend: GET /api/customers	Axios
4	Need backend	Java application receives API requests	Spring Boot
5	Need API endpoint	GET /api/customers	Spring MVC
6	Need JSON	Backend returns customer data as JSON	Jackson
7	Need database	Store customers and electricity bills	MySQL
8	Need Java ↔ database connection	Customer Java object ↔ customers table	Hibernate/JPA
9	Need validation	Check that customer name/email are valid	Validation library/framework features
10	Need your actual application rules	Calculate electricity bill based on units	YOUR BUSINESS LOGIC

11	Repetitive code	HTTP server setup, routing, JSON conversion, DB mapping, etc.	Boilerplate
12	Avoid writing repetitive code	Spring Boot/Hibernate/Jackson/etc. handle common work	Frameworks + Libraries
13	Result	You spend more time writing actual application features	Developer

So therefore ,

Framework + Libraries → Less Boilerplate → More Business Logic → Faster Development

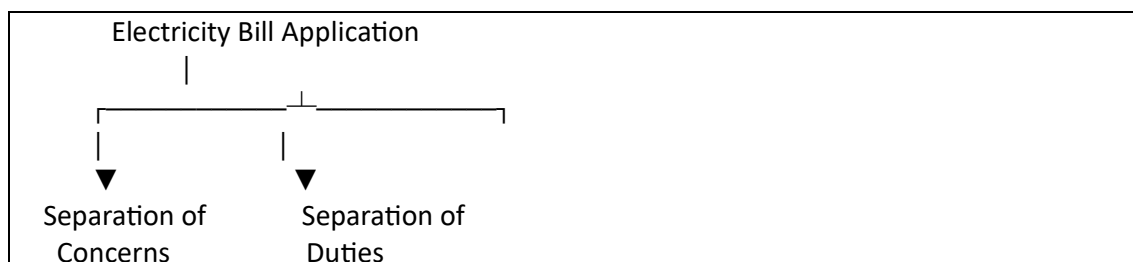
Separation of Duties or Separation of Concerns

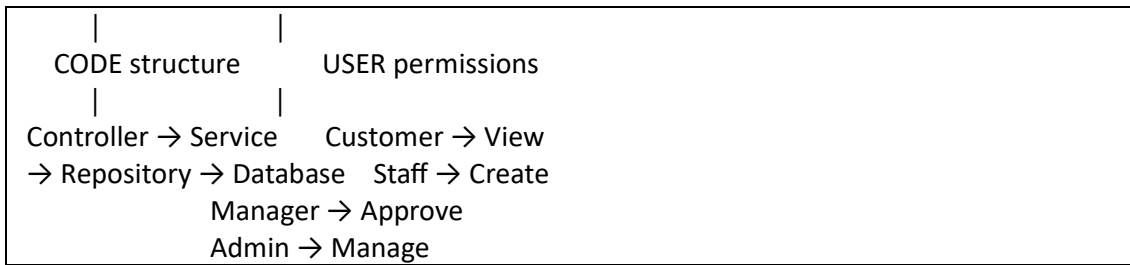
1. how is this achieved in Python (FastAPI) and Java (Springboot)

Soc = Separate The CODE Jobs.

Sod = Separate The USER Jobs/Permissions.

Architecture :





Example :

Step	Electricity Bill System	SoC / SoD
1	User logs in	Authentication
2	System identifies user as BILLING_STAFF	Authentication
3	Staff calls POST /bills	Authorization / SoD
4	Router/Controller receives request	SoC
5	Service checks bill business rules	SoC
6	Repository saves bill	SoC
7	Manager calls POST /bills/100/approve	Authorization / SoD
8	Only MANAGER is allowed	SoD
9	Controller → Service → Repository	SoC
10	Bill is stored in MySQL	SoC